ΔV Co-op Mod Spec

Design spec, v2 · written 2026-10-09, status notes as of 2026-10-10. A PDF copy of this file is SPEC.pdf.

This is a design document: it describes what the mod is meant to become. For what actually works today, see the Status section of the README.

Scope

Up to five players share one ring. Each plays their own full game, with their own save, money, cargo and story, and whenever two or more of them are near each other in the belt they see and mine the same rocks. Players can join a friend who is already out in the ring, or fly to one, and can be any distance apart.

This is v2 of the design, redrawn on 2026-10-09 after M1 proved the two-ship model. v1 (two players, guest leashed to the host's bubble) is superseded; what was built for M0 and M1 carries forward unchanged.

The mod ships as a zip in the game's `mods/` folder and runs on the retail Steam build (Godot 3.6, GodotSteam build). No patched executable, no repacked pck, no engine rebuild.

In scope

- A lobby of up to 5. Anyone can host, anyone can join at any time, including while the others are deep in the ring.
- Everyone keeps playing their own single-player game. Economy, crew, story flags, saves and Enceladus are per player and never merge.
- Friends show on the astrogation map and LIDAR at any distance. A player can plot a course to a friend from the ring, or dive to a friend's position from Enceladus.
- When players come within range of each other they share one physical world: the same asteroids, the same ore, the same NPC ships. Mining, claws, cargo, collisions, weapons fire and damage all work between them.
- When they drift apart again, each goes back to their own locally simulated bubble with no network cost.
- Whoever is simulating a shared area can leave or drop and the others carry on without losing the rocks around them.

Not in scope

- Shared economy, shared cargo holds, trading between players, or crew moving between ships.
- Dedicated servers. One of the players is always the simulator of a shared area.
- Shared bullet time or shared pause when players are apart. Each player's clock is their own until they share an area.
- Tales (DLC) content and the demo build.

Definition of done

Done means a stranger downloads it, plays a full evening with three friends, and never thinks about the mod. Every roadmap gate rolls up to one of these.

Player-visible

1. Install is a zip in `mods/` plus one launch option. A build mismatch refuses to load with a plain message.
2. Host, join and invite from the pause menu or title screen; Steam overlay invites work. No command line, no typed lobby ids.
3. Join at any time: from Enceladus by diving to a friend's marker, or from the ring by plotting a course to them with the game's own astrogation.
4. Friends show on the HUD and astrogation map at any distance with name and range.
5. Players in range see the same rocks, ore, NPCs and wrecks. A rock one player cracks breaks the same way for everyone.
6. Mining together works end to end: any player can laser, grab, claw, load and bay any rock in the shared area, and each takes home what they docked. No duplicated ore, no vanished ore.
7. Collisions and weapons between players and with shared bodies behave as in single player. Dying, rescue and exiting the ring work with friends nearby.
8. Drifting apart or a friend quitting mid-cell is seamless: no freeze, no rocks jumping, no stuck ship. Authority hands over within a second.
9. Alone, the game is the stock game: same bullet time, story and performance. Nothing of the mod shows.
10. Nothing is ever written to another player's save.

Engineering

- A four-player cell runs at 60 fps on the authority with under 250 kB/s upload, measured.
- Steam P2P proven across different accounts and networks.
- Works across screen resolutions, aspect ratios and pixel densities: everything synced is in world units, and every mod UI element (friend markers, lobby panel) is checked at 16:9, 16:10, 21:9 and 5:4, low to 4K, and at the game's UI-scale extremes.
- Two hours of four-player play with no desync report and zero mod script errors in the engine log.
- Reconnect after a drop returns a player to the same lobby and their own ship.
- A game update is handled by re-decompile, diff, re-point, bump the build pin, in about an hour, documented.
- README covers install, play, troubleshooting and known limits.

Not required for done: shared bullet time, trading, text chat, Tales DLC, dedicated servers, story-state migration (see the last section).

Decisions

Every fork is decided with a default so building can continue. The two that define v2 are the first two rows: the world is shared only where players are, and whoever has the lowest id in a shared area simulates it.

Fork	Decision	Why
World model	Local bubbles, shared cells. Each machine simulates its own asteroid bubble exactly as the stock game does. Players within 40,000 px of each other form a cell and share one simulation.	The stock spawner only ever populates around one focus object and throttles at about 2,500 bodies. One machine cannot simulate four bubbles 10 million px apart.
Cell authority	The incumbent keeps it: a player joining a cell never takes over. When two cells meet, the one with more members keeps its authority, then the older one; lowest peer id only breaks a full tie (and picks between two solo players). Others suspend their spawner, drop their own rocks inside the shared radius, and track the authority's. Authority passes to the next-lowest id if the current one leaves.	Deterministic, no election traffic, and every member already holds the full body set as puppets, so handoff is a flag flip. Changed 2026-10-25 from plain lowest-id: in testing, a late joiner with a lower id replaced an established cell's 156 rocks with its own 27.
Ship transforms	Owner-authoritative, always. Your machine decides where your ship is; everyone else runs a puppet from your intent (30 Hz) and state (20 Hz).	Proven in M1. Anything else rubber-bands the pilot.
Body ownership	Default owner is the cell authority. A body a player grabs, holds, docks or loads migrates to that player for as long as they hold it. Damage from a non-owner is sent to the owner as an event.	Mining and cargo are per player; the player doing it must feel no latency on the rock in their claw.
Coordinates	Beacons use global ring coordinates. Inside a cell every member adopts the authority's floating origin exactly and all cell traffic is origin-relative.	Godot 3 Vector2 is float32; global coordinates at 30 million px carry 2 to 4 px of error, which is fine for a map marker and not for a rock in a claw.
Transport	Full P2P mesh over Steam (one lobby, a session per pair). ENet star with server relay for the one-PC dev loop.	Compiled into the exe. A mesh avoids one player's upload carrying everyone.
NPCs and story	Each player's story events and NPC ships run locally when alone. In a cell, only the authority's run; other members suppress theirs and see the authority's NPCs as puppets.	Story scripts assume one player and one ship; suppressing is cheaper than merging them.
Time scale	Free when alone. Pinned to 1.0 for every member of a cell. Inside a cell the pause menu opens without pausing the game (alone, pause works as stock).	`Engine.time_scale` is global per machine; dilation inside a shared area desyncs it. Changed 2026-10-25 from "pause inside a cell pauses the cell": the game pauses on focus loss and the Steam overlay too, so one player alt-tabbing would freeze everyone.
Joining	At any time. From Enceladus: dive to a friend's beacon. From the ring: plot a course to a friend with the game's own astrogation, paying its time and remass.	Both reuse existing mechanics, so joining feels like part of the game.
Persistence	Per player, untouched. Nothing is written to anyone else's save.	Removes every economy and story reconciliation problem at once.
Player count	Lobby max 5, designed for 4.	Cell bandwidth scales with members; 4 in one cell is the budgeted worst case.
Mod delivery	Single zip in `<install>/mods/`, launched with `--enable-mods`.	Official loader, proven in M0.

Architecture

Every machine runs the unmodified game plus one `CoopNet` node, and when alone it is exactly the stock game. The only always-on traffic is a one-per-second beacon from each player to every other. When two or more

players come within range they form a cell: the member with the lowest id keeps simulating, the others suspend their own spawner and track the authority's bodies, and the pair exchange ship intent and state as proven in M1. Players outside every cell cost nothing and see each other only as map markers.

Game-side seams

The mod touches eight scripts. Everything else in the 489-script codebase is left alone. Paths are `res://` paths inside the pck, recovered with GDRE Tools 2.7.1.

Script	What it does today	Hook	Why this seam
`ships/ship-ctrl.gd` (5,611 lines, RigidBody2D)	One class drives player and NPC ships. `_process` calls `manualControl` when `playerControlled`, `aiControl` when `ai`, then `executeThrusterFire` applies the `thrusterShedule` dict and the `fired` dict to thrusters and weapons.	Done in M1. A third mode `netControlled`: skip both planners, write the received schedule into `thrusterShedule`, call parent `_process`; correct the transform in `_integrate_forces`. Local player ship publishes its schedule from `executeThrusterFire`.	The schedule dict is the ship's whole intent for the frame. Works for player puppets and, later, for NPC puppets.
`CurrentGame.gd`	Holds `playerShip`, `globalOffset`, `targetTimeScale`, `state.ship` (the serializable ship config), `diveTarget`.	Extension. Clamp `targetTimeScale` to 1.0 while in a cell; expose origin adoption (set `globalOffset` to the authority's and shift every body by the delta in one physics frame).	The ship config dict already round-trips through the save, so it round-trips through the wire.
`BulletTimer.gd`	Drives `Engine.time_scale` toward `CurrentGame.targetTimeScale` every physics frame.	Extension. Hold `Engine.time_scale = 1.0` while in a cell.	Single owner of the global clock.
`AsteroidSpawner.gd` (the `AsteroidField` node)	Streams 500 to 2,500 bodies around `focusObject` via `spawnAsteroidByClass` / `spawnByBlueprint`, despawns by distance (`despawnFarAway`, two DespawnObserver areas), slipstreams the origin past 2,097,000 px.	Extension. Authority: tag every body with a net id, broadcast spawn/despawn, stream transforms by tier. Member: `spawnThings = false`, despawn suppressed for tracked bodies, bodies created only from authority messages. Alone: untouched.	The spawner is the only producer of world objects; its on/off switch is the cell boundary.
`asteroids/asteroid.gd`	RigidBody2D. Fracture at line 215 calls `field.spawnAsteroidByClass` for shards. `applyKineticDamage` / `applyEnergyDamage` are the damage entry points.	Owner only fractures. Non-owner: damage entry points become "send damage event to owner". Shards arrive as spawns.	Fracture uses `rand_range`; only one machine may roll it.

Script	What it does today	Hook	Why this seam
`ships/modules/DockingArm.gd`, `Bay.gd`, cargo paths in `ship-ctrl`	Claw grab, cargo hold entry (`processCargoBodyEntry`), mass driver loading.	Extension. On grab or hold entry, request ownership of the body; on release, hand it back to the authority.	"Touch to own" keeps the rock in your claw latency-free.
`story/*` managers, `Factions.gd`	Spawn pirates, rescues, ring storms, NPC traffic relative to `getPlayerShip()`.	Extension on the few spawn entry points: no-op while a non-authority cell member. NPC ships spawned by the authority are announced like player ships and puppeted.	Suppression is one check per spawn site.
`hud/LIDAR.gd`, `enceladus/Map/*`, `hud/AutopilotOverlay.gd`	Draw contacts and the astrogation map around the player.	Additive: draw friend beacons as markers, add "plot course to friend" and "dive to friend" entries.	Join-in-the-belt UI.
`Game.gd`	Builds the dive, places the player ship, handles exit and game-over.	Extension. Enter/leave dive hooks for beacons; dive target from a friend's beacon.	Only place with correct cutscene and zone flags.
`weapons/massdriver.gd`, `weapons/projectiles/projectile.gd`, `weapons/laser.gd`	Massdriver spawns a RigidBody2D bullet with `shooter`; laser is a raycast.	Bullets are owned by their shooter and replicated as bodies; laser hits on non-owned rocks become damage events.	Bullets are physics bodies that damage rocks.
`ModLoader.gd`	Loads zips from `<install>/mods/` with `--enable-mods`; `installScriptExtension` takes over a script path with a subclass.	Entry point. Proven in M0.	Official API.

Not touched: `enceladus/*` beyond the dive-target picker (the station stays single-player), `comms/*` (NPC dialog runs on whichever machine owns the NPC), save and economy code anywhere.

Replication model

Two rules. Outside a cell, nothing is replicated but beacons. Inside a cell, every body has exactly one owner, the owner's simulation is truth, and the owner is the cell authority unless a player is holding the body.

Entity	Owner	Wire payload	Rate	Receiver-side handling
Beacon (every player, always)	Each player	global position, velocity, heading, transponder, ship name, zone (rings / transit / station)	1 Hz, unreliable, to everyone	Drives friend markers on map and LIDAR, the join targets, and cell formation by distance.

Entity	Owner	Wire payload	Rate	Receiver-side handling
Player ship, in range (under 40,000 px)	Owner	intent: `thrusterShedule` as {thruster: power, vector}, `fired` as {system: power}. state: origin-relative position, rotation, velocities, damage deltas	intent 30 Hz, state 20 Hz, unreliable; damage reliable	`netControlled` puppet, velocity-corrected toward state, snap past 400 px. Spawned when a beacon enters range, freed when it leaves. Proven in M1.
Cell membership and origin	Each member computes it from beacons; the authority confirms	cell id, member list, authority id, authority's `globalOffset` and origin generation	Reliable, on change	On joining a cell a member shifts its own ship and everything it keeps to the authority's origin in one physics frame, then disables its spawner.
Asteroids and ore in a cell	Authority (or holder)	spawn: net id, class, vein, scale, mass, position, velocity. update: net id, position, rotation, velocities. despawn: net id. owner-change: net id, new owner	Spawn, despawn, owner-change reliable. Updates per receiver: 10 Hz within 6,000 px of that receiver's ship, 2 Hz to 30,000 px, none beyond	Tracked RigidBody2D with the correction integrator; never spawns, fractures or despawns on its own. Bodies beyond the receiver's bubble are not created at all.
Damage to a body you don't own	Non-owner sends, owner applies	target net id, kind (kinetic / energy), amount, point	Reliable, event	Owner applies through the stock `applyKineticDamage` / `applyEnergyDamage`, rolls fracture, broadcasts shards.
Held bodies (claw, hold, loader)	The holding player	owner-change on grab and release, then the holder streams it like its own ship	Reliable change, 20 Hz state	Latency-free handling for the player doing the work; the rock returns to the authority on release.
NPC ships and drones in a cell	Authority	same as player ships, plus faction and transponder on spawn	Spawn reliable, intent and state 10 Hz	Puppet path shared with player ships. Non-authority members suppress their own story and NPC spawns.
Projectiles	Shooter	spawn: net id, shooter, position, velocity; despawn on impact	Reliable spawn, no updates	Receiver simulates ballistic flight locally. Laser beams come from the shooter's `fired` dict.
Time scale and pause	Each player	none	n/a	Every cell member pins `Engine.time_scale` to 1.0; the pause menu doesn't pause the tree in a cell. Alone, untouched.
Authority handoff	Next-lowest id	authority-change: cell id, new authority, origin generation	Reliable	New authority flips its tracked bodies to simulated and starts streaming; others re-point. Bodies the old authority never broadcast are gone, which is indistinguishable from normal despawn.

Bandwidth budget. Outside a cell a player sends about 100 bytes per second. Inside a cell the authority's upload is the cost: per receiver, about 300 near bodies at 10 Hz plus 2,200 far bodies at 2 Hz is roughly 160 kB/s before quantization. With 16-bit position deltas and 8-bit velocities that halves. Four members in one cell is the budgeted worst case at about 250 kB/s authority upload after quantization. The first cut if it is too high: far tier at 0.5 Hz and only class 0 to 2 rocks beyond 12,000 px.

Identity. A net id is (owner peer id, 32-bit counter), so two machines can mint ids without coordinating. Shards from fracture get fresh ids from the fracturing owner. A lookup dict on every machine maps id to node. Bodies whose owner disconnects are re-owned by the authority if it holds them, otherwise freed.

Session flow

There is no shared "dive". Each player starts and ends dives on their own schedule; the lobby only carries beacons until two players come within range. A cell forms and dissolves from distance alone, with 5,000 px of hysteresis so a pair flying at the edge does not flap. Joining is either a dive from Enceladus targeted at a friend's beacon, or a course plotted to a friend from the ring; both put the joiner in range and the cell does the rest.

Mod packaging and dev loop

The deliverable is one zip, `dv-coop.zip`, dropped into `<install>/mods/` and run with `--enable-mods` in the Steam launch options. The loader mounts the zip as a resource pack, finds `modmain.gd`, instantiates it under the `ModLoader` autoload, and the mod installs its script extensions before any scene loads.

Zip layout

```
dv-coop.zip
  dv-coop/
    modmain.gd              # MOD_PRIORITY, installs extensions, adds CoopNet node
    net/CoopNet.gd          # transport, lobby, packet codec, tick loop
    net/Codec.gd            # quantization, message types, sequence numbers
    net/Registry.gd         # netId <-> node lookup, origin generation
    ext/ship-ctrl.gd        # extends "res://ships/ship-ctrl.gd"
    ext/CurrentGame.gd      # extends "res://CurrentGame.gd"
    ext/BulletTimer.gd      # extends "res://BulletTimer.gd"
    ext/AsteroidSpawner.gd  # extends "res://AsteroidSpawner.gd"
    ext/asteroid.gd         # extends "res://asteroids/asteroid.gd"
    ext/Game.gd             # extends "res://Game.gd"
    ext/massdriver.gd       # extends "res://weapons/massdriver.gd"
    ui/Lobby.tscn           # host / join panel on the title screen and pause layer
```

Script extension rule. Each `ext/*.gd` file starts with `extends "res://<original path>"`, overrides only the functions named in the seams table, and calls the parent with `.function()` for everything else. `ModLoader.installScriptExtension` then takes over the original path, so every scene that instances the original script gets the subclass. Overriding `_process` or `_physics_process` must call the parent version explicitly or the ship stops working.

Transport layer. `CoopNet` wraps GodotSteam's `createLobby`, `joinLobby`, `sendP2PPacket`, `readP2PPacket` and the `lobby_created`, `lobby_joined`, `p2p_session_request` signals. A `--coop-enet=<host:port>` command-line flag swaps in `NetworkedMultiplayerENet` so two instances on one PC can test without two Steam accounts.

Two-instance testing. Steam allows one running instance per account. Dev loop is: a second Windows user account with its own Steam login for real P2P tests, and the ENet flag for the daily loop on one account. Both

instances need `--enable-mods` and a `user://` override so saves do not collide; Godot 3 honors `--main-pack` and a separate `APPDATA`, so launch the second instance with a different `APPDATA` environment variable.

Platform. Develop on Windows, where the install and the test rig already are. The mod is GDScript in a zip, identical on every platform, and the game ships a native Linux build with the same GodotSteam module. A Steam Deck or a spare Linux box makes a real second machine for P2P tests without a second Windows license; GDRE Tools also has a Linux release if the recovery ever needs to run there.

Rebuilding the source for reference. The decompiled project is regenerated with:

```
gdre_tools.exe --headless --recover="C:\Program Files (x86)\Steam\steamapps\common\dV Rings of
Saturn\Delta-V.pck" --output=<dir>
```

It takes about five seconds and produces an editable Godot 3.6 project. The mod is developed in a plain folder, zipped by a script, and never checked into the game directory. A game update that changes a hooked function shows up as a decompile diff against the previous recovery.

Editor. Godot 3.6 standard editor can open the recovered project for reading and for building `ui/Lobby.tscn`. GodotSteam calls only resolve inside the retail exe, so the `Steam` singleton is guarded with `Engine.has_singleton("Steam")` to keep the editor from erroring.

Work plan

Each diamond is a gate: the next milestone does not start until the criterion under it has been seen on two or more machines. M3 and M4 are the long poles. M2 reuses M1 wholesale and mostly adds the mesh and the map. Weeks assume one person working most evenings and weekends; M0 and M1 both landed in one day, so the estimates are conservative.

Status, 2026-10-09: M0 and M1 done. M0 passed: the mod loads through the official loader, the ship-controller extension is live on real ships, a Steam lobby is created from a direct exe launch, and packets flow between two instances at about 16 ms round trip. M1 works on the two-instance ENet loop: both machines auto-enter one dive, exchange ship configs, spawn a puppet of each other's ship, and agree on separation to within 2 px at about 10 kB/s each way. Open: ten minutes of hand-flown formation, and a Steam P2P packet between two different Steam accounts.

M2 status, 2026-10-09: mesh, beacons and markers working on three instances. Peers are a dictionary rather than a host/guest pair, the ENet dev transport relays through the server, every peer beacons once a second, puppets appear under 40,000 px and drop past 45,000 px, and a friend-marker overlay draws a chevron, name and distance at the screen edge for every peer at any range. An out-of-range peer costs about 1 kB/s. The gate (three instances, markers at any distance) was confirmed by screen capture. Open: a hand-flown approach from out of range into range.

Risks and first measurements

The cell model trades one hard problem (one machine simulating everyone's bubble) for three medium ones: the moment a cell forms, body ownership while mining, and the authority leaving. All three are measurable on

three instances on one PC.

Risk	What goes wrong	Measure first	Fallback
Cell formation is visible	When a member joins a cell, its own rocks inside the shared radius vanish and the authority's appear. A player looking at a specific rock sees it swap.	Form a cell at 40,000 px, where the joiner can barely see anything of the other side, and measure how many rocks inside the joiner's view (about 6,000 px) actually change.	Fade: keep the joiner's local rocks as non-colliding ghosts for 2 s while the authority's stream in, then free them.
Authority upload	Four members in one cell push the authority past what its connection carries, or packet decoding stalls its main thread.	Three instances on one PC, full body stream with tiers and quantization, read the authority's upload rate and frame time.	Far tier at 0.5 Hz; cap tracked bodies per receiver at 1,200; the authority is the member with the best measured upload rather than the lowest id.
Ownership races	Two players grab the same rock in the same tick, or a claw grabs a rock mid-fracture.	Script: two puppets claim one net id simultaneously; assert exactly one owner after one round trip.	The authority arbitrates claims in order of arrival; a losing claw releases.
Origin adoption	Shifting a member's world to the authority's origin in one frame moves 2,500 bodies, particles, and the camera; a missed node jumps by millions of px.	Unit-test the shift on a full bubble; grep every node type that stores a position outside the scene tree (debris fields, LIDAR history, autopilot targets).	Adopt the origin only when the member's spawner is already suspended and its bubble is small.
Story suppression misses a spawn site	A non-authority member spawns pirates nobody else sees, which shoot a ship everyone else sees.	List every `spawnByBlueprint` / `createShipBy*` call site outside the spawner (about 40) and gate each.	Treat any ship a member spawns as a member-owned replicated NPC instead of suppressing.
Script extension coverage	A scene that preloaded the original script before the mod ran keeps the old class.	Proven for `ship-ctrl`; re-verify per extension with the self-test pattern from M0.	`set_script` at instance time.
Game update breaks a hook	A renamed function silently drops a hook.	Pin the mod to a game build; refuse to load on mismatch.	Re-decompile, diff, re-point. About an hour per update.
Steam P2P never tested	Everything so far ran over ENet on one PC.	First session with a second Steam account.	ENet with port forwarding.

Unknowns to resolve in code, not in this spec

- Whether tracked bodies with the correction integrator keep collision feel acceptable, or whether members need kinematic bodies plus owner-side collision events.
- How the stock despawn observers behave with 40,000 px of tracked bodies that the member did not spawn.
- Whether `DockingArm` grab detection fires reliably on a tracked body before the ownership transfer lands.

Out of scope now, v2 candidates

Ordered by how much a four-player session will want them.

1. Shared bullet time inside a cell: the authority broadcasts a time-scale event and every member dilates together.
2. Trading: hand ore or remass between ships through the docking arm.
3. Text chat in the pause layer; voice stays on Steam or Discord.
4. Host migration of a whole cell's story state, so pirates mid-fight survive the authority leaving.
5. Tales (DLC) content.